# Homework

Mehreen Ali Gillani

2026-05-06

**Introduction:**

Imagine 10000 receipts sitting on your table. Each receipt represents a transaction with items that were purchased. The receipt is a representation of stuff that went into a customer's basket - and therefore 'Market Basket Analysis'.

That is exactly what the Groceries Data Set contains: a collection of receipts with each line representing 1 receipt and the items purchased. Each line is called a transaction and each column in a row represents an item. The data set is attached.

Your assignment is to use R to mine the data for association rules. You should report support, confidence and lift and your top 10 rules by lift.

Extra credit: do a simple cluster analysis on the data as well. Use whichever packages you like.

### 1. Load the Groceres dataset

```r
# ================================
# Market Basket Analysis & Clustering
# Groceries Dataset
# ================================

# 1. Load required packages and dataset
# install.packages(c("arules", "arulesViz", "factoextra", "cluster", "pheatmap"))
library(arules)        # For association rules
```

```
## Loading required package: Matrix

##
## Attaching package: 'arules'

## The following objects are masked from 'package:base':
##
##     abbreviate, write
```

```r
library(arulesViz)     # For visualizing rules (optional)
library(cluster)       # For clustering
library(factoextra)    # For visualizing clusters
```

```
## Loading required package: ggplot2

## Welcome to factoextra!

## Want to learn more? See two factoextra-related books at https://www.datanovia.com/en/product/practica
```

```r
library(pheatmap)
```

```r
data("Groceries")
summary(Groceries)
```

```
## transactions as itemMatrix in sparse format with
##  9835 rows (elements/itemsets/transactions) and
##  169 columns (items) and a density of 0.02609146
##
## most frequent items:
##       whole milk other vegetables       rolls/buns             soda
##             2513             1903             1809             1715
##           yogurt          (Other)
##             1372            34055
##
## element (itemset/transaction) length distribution:
## sizes
##    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15   16
## 2159 1643 1299 1005  855  645  545  438  350  246  182  117   78   77   55   46
##   17   18   19   20   21   22   23   24   26   27   28   29   32
##   29   14   14    9   11    4    6    1    1    1    1    3    1
##
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   2.000   3.000   4.409   6.000  32.000
##
## includes extended item information - examples:
##        labels  level2           level1
## 1 frankfurter sausage meat and sausage
## 2     sausage sausage meat and sausage
## 3  liver loaf sausage meat and sausage
```

**Dataset overview:**

- This dataset has 9835 transactions (receipts) and 169 items
- Average basket size is 4.4 items, with most baskets containing 1-6 items (median = 3 items).
- The most frequently purchased item is "whole milk" (appearing in 2,513 baskets, 25.6% of transactions), followed by other vegetables, rolls/buns, soda, and yogurt.

### 2. Mine association rules

```r
# Use Apriori algorithm with reasonable thresholds
# (adjust support/confidence for meaningful rules)
rules <- apriori(Groceries,
                 parameter = list(supp = 0.001,    # support threshold (0.1%)
                                  conf = 0.5,      # confidence threshold (50%)
                                  target = "rules",
                                  minlen = 2))     # at least 2 items per rule
```

```
## Apriori
##
## Parameter specification:
##  confidence minval smax arem  aval originalSupport maxtime support minlen
##         0.5    0.1    1 none FALSE            TRUE       5   0.001      2
##  maxlen target  ext
##      10  rules TRUE
```

```
##
## Algorithmic control:
##  filter tree heap memopt load sort verbose
##     0.1 TRUE TRUE  FALSE TRUE    2    TRUE
##
## Absolute minimum support count: 9
##
## set item appearances ...[0 item(s)] done [0.00s].
## set transactions ...[169 item(s), 9835 transaction(s)] done [0.00s].
## sorting and recoding items ... [157 item(s)] done [0.00s].
## creating transaction tree ... done [0.00s].
## checking subsets of size 1 2 3 4 5 6 done [0.00s].
## writing ... [5668 rule(s)] done [0.00s].
## creating S4 object  ... done [0.00s].
```

```r
# 4. Sort rules by lift (highest first)
rules_by_lift <- sort(rules, by = "lift", decreasing = TRUE)
```

**Apriori algorithm Output:**

- The Apriori algorithm generated 5,668 association rules using minimum support of 0.001 (at least 9 transactions) and confidence of 0.5.
- The algorithm successfully processed all 9,835 transactions with 169 items, reducing to 157 items after recoding (excluding items too rare to meet the support threshold).
- Rules include itemsets from size 2 up to size 6 as specified by maxlen=10.

### 3. Display top 10 rules by lift with support, confidence, lift

```r
# ========== TOP 10 RULES BY LIFT ==========
inspect(rules_by_lift[1:10])
```

```
##      lhs                          rhs                  support confidence    coverage     lift count
## [1]  {Instant food products,
##       soda}                   => {hamburger meat} 0.001220132  0.6315789 0.001931876 18.99565    12
## [2]  {soda,
##       popcorn}                => {salty snack}    0.001220132  0.6315789 0.001931876 16.69779    12
## [3]  {flour,
##       baking powder}          => {sugar}          0.001016777  0.5555556 0.001830198 16.40807    10
## [4]  {ham,
##       processed cheese}       => {white bread}    0.001931876  0.6333333 0.003050330 15.04549    19
## [5]  {whole milk,
##       Instant food products}  => {hamburger meat} 0.001525165  0.5000000 0.003050330 15.03823    15
## [6]  {other vegetables,
##       curd,
##       yogurt,
##       whipped/sour cream}     => {cream cheese }  0.001016777  0.5882353 0.001728521 14.83409    10
## [7]  {processed cheese,
##       domestic eggs}          => {white bread}    0.001118454  0.5238095 0.002135231 12.44364    11
## [8]  {tropical fruit,
##       other vegetables,
##       yogurt,
##       white bread}            => {butter}         0.001016777  0.6666667 0.001525165 12.03058    10
## [9]  {hamburger meat,
##       yogurt,
```

```
##       whipped/sour cream}     => {butter}         0.001016777  0.6250000 0.001626843 11.27867    10
## [10] {tropical fruit,
##       other vegetables,
##       whole milk,
##       yogurt,
##       domestic eggs}          => {butter}         0.001016777  0.6250000 0.001626843 11.27867    10
```

**TOP 10 RULES BY LIFT, Observation**

- The highest lift rule (18.99) shows that buying "Instant food products" and "soda" together makes customers 19 times more likely to buy "hamburger meat" than buying it randomly.
- All top 10 rules have lift values above 11, indicating strong positive associations, with 8 of the 10 rules involving dairy products (butter, cream cheese, yogurt) or meat items (hamburger meat, ham).
- Most rules have low support (0.001-0.002 or 10-19 transactions) but high confidence (50-67%), meaning these are niche but reliable purchasing patterns.

### 4. Extra Credit: Cluster Analysis on Items

```r
# Create binary matrix (items x transactions)
item_matrix <- t(as(Groceries, "matrix"))

# Jaccard distance & hierarchical clustering
item_dist <- dist(item_matrix, method = "binary")
hc_items <- hclust(item_dist, method = "average")

# Cut into 5 clusters
clusters <- cutree(hc_items, k = 5)

# Display results
cat("\n========== ITEM CLUSTERS (k=5) ==========\n")
```

```
##
## ========== ITEM CLUSTERS (k=5) ==========
```

```r
for (i in 1:5) {
  items_in_cluster <- names(clusters[clusters == i])
  cat(paste0("\nCluster ", i, " (", length(items_in_cluster), " items):\n"))
  cat(paste(items_in_cluster[1:min(10, length(items_in_cluster))], collapse = ", "))
  if (length(items_in_cluster) > 10) cat(", ...")
  cat("\n")
}
```

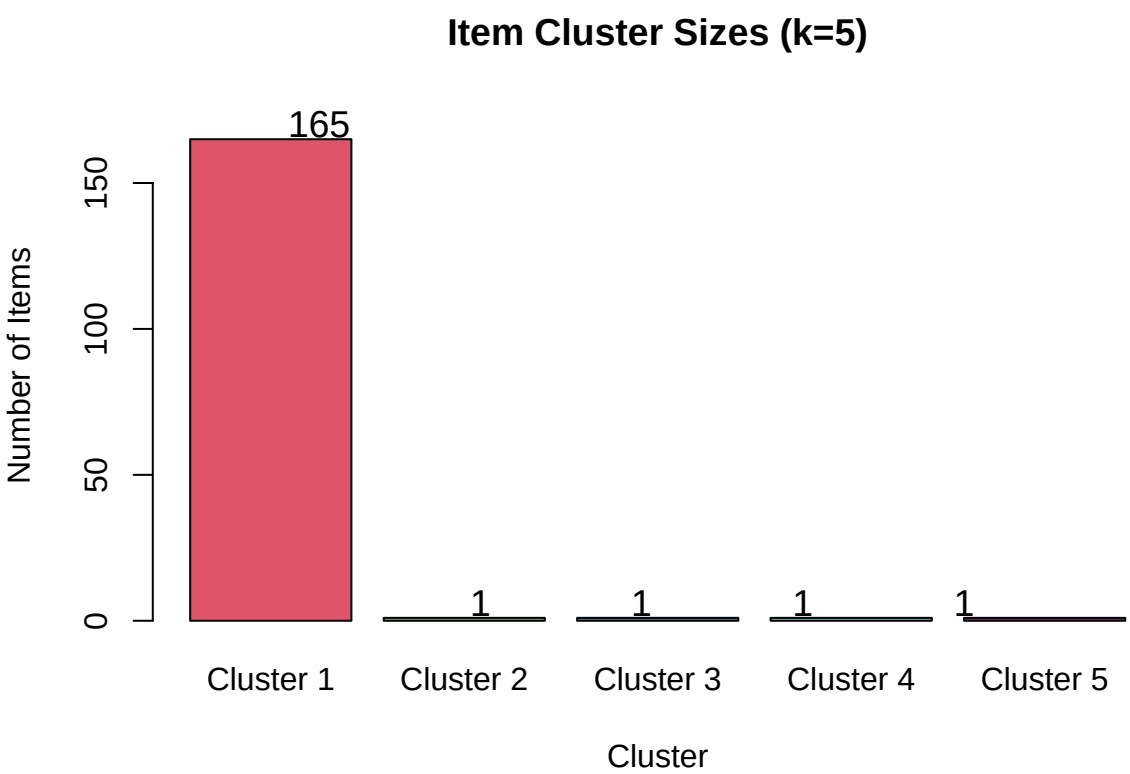
```
##
## Cluster 1 (165 items):
## frankfurter, sausage, liver loaf, ham, meat, finished products, organic sausage, chicken, turkey, po
##
## Cluster 2 (1 items):
## frozen chicken
##
## Cluster 3 (1 items):
## liqueur
##
## Cluster 4 (1 items):
## make up remover
```

```
##
## Cluster 5 (1 items):
## sound storage medium
```

```r
# bar plot of cluster sizes
cluster_sizes <- table(clusters)

barplot(cluster_sizes,
        main = "Item Cluster Sizes (k=5)",
        xlab = "Cluster",
        ylab = "Number of Items",
        col = 2:6,
        names.arg = paste("Cluster", 1:5),
        ylim = c(0, max(cluster_sizes) + 10))

# Add count labels on bars
text(x = 1:5, y = cluster_sizes + 5,
     labels = cluster_sizes, cex = 1.2)
```




**Interesting observation: cluster 1 has 165 items whereas cluster 2-5 has one item each.**

**Lets check those items frequency and figure out why its happening?**

```r
# Check frequency of singleton items
itemFrequency(Groceries)[c("frozen chicken", "liqueur",
                           "make up remover", "sound storage medium")]
```

```
##       frozen chicken              liqueur      make up remover
##         0.0006100661         0.0009150991         0.0008134215
## sound storage medium
##         0.0001016777
```

**Why 1 cluster has 165 items, others have 1 each**

**Frequencies of the 4 "singleton" items:**

- frozen chicken → appears in 6 receipts (0.06%)
- liqueur → appears in 9 receipts (0.09%)
- make up remover → appears in 8 receipts (0.08%)
- sound storage medium → appears in 1 receipt (0.01%) The math: For Jaccard distance:

If an item appears in only 1-9 transactions out of 9835

It almost never co-occurs with any other item

Intersection with any other item = 0 (or very tiny)

Jaccard similarity = 0 → distance = 1 (maximum)

*These items are statistical orphans - they don't cluster because they don't share baskets with others.*

**Filter out rare items:**

```r
# Function to get cluster summary with different threshold
cluster_summary <- function(min_support, n_clusters = 4) {
  # Filter items
  keep_items <- itemFrequency(Groceries) >= min_support
  data_filt <- Groceries[, keep_items]

  # Cluster
  mat <- t(as(data_filt, "matrix"))
  dist_mat <- dist(mat, method = "binary")
  hc <- hclust(dist_mat, method = "average")
  clusters <- cutree(hc, k = n_clusters)

  # Return clean results
  list(
    threshold = paste0(min_support * 100, "%"),
    transactions = paste0(min_support * 9835, "+"),
    items = ncol(data_filt),
    cluster_sizes = as.vector(table(clusters)),
    clusters = clusters
  )
}

# Test different thresholds
thresholds <- c(0.01, 0.005, 0.001)  # 1%, 0.5%, 0.1%
n_transactions <- c(98, 49, 10)

# output
for(i in 1:3) {
  result <- cluster_summary(thresholds[i])
  cat("Threshold:", result$threshold,
      "| Min transactions:", n_transactions[i],
      "| Items kept:", result$items, "\n")
  cat("  Cluster sizes:", paste(result$cluster_sizes, collapse = ", "), "\n\n")
}
```

```
## Threshold: 1% | Min transactions: 98 | Items kept: 88
```

```
##   Cluster sizes: 81, 2, 3, 2
##
## Threshold: 0.5% | Min transactions: 49 | Items kept: 120
##   Cluster sizes: 117, 1, 1, 1
##
## Threshold: 0.1% | Min transactions: 10 | Items kept: 157
##   Cluster sizes: 150, 3, 2, 2
```

**Key insight:**

- 1% threshold (98+ transactions) → Most balanced (81 items in main cluster, small outliers)
- 0.5% or 0.1% → Highly skewed because rare items become singletons

**Recommendation:** Use min_support = 0.005 (49+ transactions) for meaningful clusters without too many outliers.

---